

HelixTerminator

HelixTerminator

La plataforma de terminal de confianza cero para equipos: cada sesión SSH, segura, compartida y asistida por AI.

Resumen

HelixTerminator es una plataforma empresarial de terminales y gestión de sesiones SSH, desarrollada como un sistema de microservicios Go con clientes Flutter multiplataforma. Gestiona, registra y protege sesiones remotas bajo un modelo de confianza cero, incorpora colaboración en tiempo real y añade asistencia AI directamente en la terminal.

Descripción breve

HelixTerminator es una plataforma empresarial de terminales y gestión de sesiones SSH basada en confianza cero: un backend de microservicios Go junto con clientes Flutter para seis plataformas. Administra hosts, gestiona conexiones, registra sesiones, permite la colaboración en tiempo real e integra ayuda asistida por AI para comandos, explicación de resultados y respuesta a incidentes.

Descripción detallada

HelixTerminator es una plataforma empresarial de terminales y acceso remoto, estructurada en dos módulos —una Plataforma de Terminales y un Broker de Conexiones— implementados mediante un catálogo de microservicios Go con un único cliente Flutter compatible con seis plataformas. Su objetivo es eliminar por completo las herramientas ad hoc de SSH: nada de clientes por equipo, proliferación de claves privadas ni lagunas en las

auditorías, reemplazados por un sistema único, gobernado, auditable y colaborativo que trata el acceso remoto como infraestructura, no como un hábito personal.

El backend gestiona todo el ciclo de vida del acceso remoto de principio a fin. Los hosts y grupos se administran mediante cadenas de bastiones o *jump hosts*; un proxy SSH gestiona la autenticación mediante contraseñas, claves públicas y certificados; un proxy de E/S de terminal transmite la sesión a través de WebSocket; SFTP maneja transferencias reanudables; y se incluyen reenvío de puertos, gestión de fragmentos y espacios de trabajo, así como grabación de sesiones en formato *asciinema* firmado, que permite reproducirlas con total confianza. La seguridad se basa en el principio de confianza cero por diseño, no como una idea posterior: un almacén proporciona almacenamiento de secretos con conocimiento cero, un servicio PKI emite certificados SSH de corta duración para que no queden credenciales permanentes expuestas al robo, los llaveros respaldados por hardware (Secure Enclave / Android Keystore / DPAPI / HSM) mantienen las claves fuera del disco, FIDO2/ WebAuthn y OIDC/SAML gestionan la autenticación, y un registro de auditoría solo anexasible y encadenado mediante Merkle genera pruebas a prueba de manipulaciones para SOC 2 e ISO 27001. Además, la colaboración en tiempo real permite que varios operadores compartan una misma sesión en vivo con roles de observador, copiloto o propietario, manteniendo la coherencia mediante sincronización de buffers con CRDT.

Un servicio AI se integra sobre la terminal, añadiendo autocompletado de comandos, explicaciones en lenguaje sencillo de los resultados, detección de anomalías, generación de guías de procedimientos (*runbooks*) y asistencia práctica durante incidentes, transformando la terminal de un simple conducto en un asistente en los momentos clave. Toda la plataforma es nativa para contenedores —Kubernetes, Helm, Terraform— e incluye un stack completo de observabilidad con OpenTelemetry, Grafana, Jaeger y Loki. Además, se integra con el ecosistema Helix a través de un puente HelixTrack y un agente local HelixLLM. Todo el sistema opera bajo el Constitution de Helix, con compuertas de verificación de herencia anti-fraude.

Por qué lo creamos

Los equipos gestionan infraestructuras remotas mediante clientes SSH dispersos, sin un registro de auditoría compartido, sin un manejo coherente de secretos y sin forma de colaborar en tiempo real durante un incidente. HelixTerminator se desarrolló para convertir el acceso remoto en una plataforma gobernada, de confianza cero y nativa para equipos, en lugar de una herramienta por portátil.

Por qué cambia las reglas del juego

Consolida toda una lista de adquisiciones en una sola plataforma. El cliente SSH, la bóveda de secretos, la capa de bastión/PKI, la grabación de sesiones, la auditoría de cumplimiento y la colaboración en vivo son elementos que los equipos suelen comprar, integrar y reconciliar por separado, cada uno con sus propias lagunas en los puntos de unión. HelixTerminator los ofrece como un sistema gobernado único y, además, hace algo que ninguna de esas herramientas logra por sí sola: incorpora una capa AI directamente sobre el terminal que explica resultados desconocidos y elabora guías de procedimientos *mientras el incidente está activo*. La capacidad que antes no era viable es una sesión de acceso remoto que es simultáneamente segura bajo el modelo de confianza cero, grabada con evidencia de manipulación, compartida en vivo entre operadores y asistida por AI —todo a la vez, desde una sola interfaz.

Qué tiene de innovador

- **Diseño de doble módulo** (Plataforma de Terminal + Agente de Conexiones) coordinados a través de un registro de servicios, de modo que la plataforma y la capa de intermediación escalan y evolucionan de forma independiente.
- **Seguridad de confianza cero de extremo a extremo:** certificados SSH de corta duración emitidos por PKI, una bóveda de conocimiento cero, cadenas de claves respaldadas por hardware y un registro de auditoría encadenado con Merkle —sin credenciales permanentes ni rastros no verificables.
- **Colaboración en tiempo real en sesiones** con sincronización de búfer basada en CRDT y roles explícitos de observador, copiloto y propietario, lo que permite que varios operadores trabajen en un mismo terminal sin interferirse.
- **Operaciones asistidas por AI** integradas en el terminal en vivo: autocompletado, explicación de resultados, detección de anomalías y asistencia para guías de procedimientos e incidentes justo donde el operador lo necesita.
- **Cliente Flutter multiplataforma** que impulsa seis plataformas desde una única base de código, garantizando que las experiencias de escritorio, móvil y web mantengan coherencia absoluta.

Principales desafíos técnicos y cómo los resolvimos

- **Asegurar el acceso remoto sin credenciales permanentes que puedan robarse** —las claves de larga duración son el

clásico punto débil en las brechas de seguridad (vector). Se resolvió con un servicio PKI que emite certificados SSH de corta duración bajo demanda, una bóveda de conocimiento cero que almacena secretos que el propio servidor no puede leer, y almacenamiento de claves respaldado por hardware (Secure Enclave / Android Keystore / DPAPI / HSM), de modo que el material privado nunca queda expuesto en disco.

- **Permitir que varios operadores controlen una misma sesión sin corromper el búfer** —la edición concurrente en un terminal compartido es un problema complejo de consistencia. Se resolvió con sincronización de búfer basada en CRDT, elegida sobre la transformación operacional (según ADR-006) precisamente porque los CRDT convergen sin necesidad de un árbitro central.
- **Hacer que las evidencias de cumplimiento sean imposibles de alterar en silencio** —un registro de auditoría editable no prueba nada. Se resolvió con un registro de solo anexión encadenado con Merkle, donde cualquier manipulación rompe la cadena de hashes, generando evidencias exportables para SOC 2 / ISO 27001 / FedRAMP.
- **Ofrecer una experiencia de usuario coherente en escritorio, móvil y web sin mantener tres bases de código** —se resolvió con un único cliente Flutter/Dart basado en el patrón BLoC, optando por Flutter en lugar de Electron (según ADR-001) para abarcar seis plataformas desde una única fuente de verdad.

Pila tecnológica

- **Go microservicios** — la flota de backend (SSH proxy, terminal, vault, PKI, auditoría y más); seleccionada por su modelo de concurrencia y su reducido consumo de recursos en tiempo de ejecución, ideal para servicios que manejan numerosas sesiones de streaming de larga duración simultáneamente (ADR-002: Go frente a Rust/Node).
- **Flutter / Dart (BLoC)** — un único código base para seis plataformas, con BLoC para mantener el estado predecible; se optó por Flutter en lugar de Electron (ADR-001) para evitar mantener front-ends nativos y web por separado.
- **PostgreSQL** — el almacén de datos principal, elegido frente a CockroachDB (ADR-004) por su núcleo transaccional maduro y bien comprendido.
- **Kafka + RabbitMQ** — la capa de mensajería y streaming que transporta segmentos de sesión y eventos (ADR-003), combinando un registro duradero con colas flexibles.
- **Redis** — almacena los búferes de desplazamiento de la terminal y el estado activo de las sesiones, donde el acceso de baja latencia es más importante que la durabilidad.
- **SPIFFE/SPIRE + mTLS** — emite identidades criptográficas para cargas de trabajo (ADR-005), de modo que el tráfico entre

servicios se autentica mutuamente, extendiendo el principio de confianza cero dentro de la malla, no solo en el perímetro.

- **Ed25519 (EdDSA)** — firma JWT y grabaciones de sesiones (ADR-009), proporcionando firmas modernas y rápidas que hacen verificables las sesiones grabadas.
- **Kubernetes + Helm + Terraform** — despliegue nativo en contenedores con infraestructura reproducible y controlada por versiones (ADR-007/008).
- **OpenTelemetry, Grafana, Jaeger, Loki** — la pila de observabilidad para trazas, métricas, paneles e registros; **Falco, Trivy, Cosign, Sealed Secrets** — detección de amenazas en tiempo de ejecución, escaneo de imágenes, firma de artefactos y entrega cifrada de secretos en toda la cadena de suministro.

Estado y notas de transparencia

- **Estado: beta.** Un código base sustancial y en desarrollo activo (creado el 04-07-2026). Las cifras de especificación numérica en el paquete de investigación del MVP del proyecto (recuentos de endpoints, tablas y servicios) son objetivos de diseño/especificación extraídos de docs/research/mvp/, no confirmados como implementados en su totalidad, y se presentan aquí como alcance arquitectónico en lugar de métricas ya desplegadas. Las afirmaciones sobre latencia/SLO y “preparación para producción” no han sido verificadas de forma independiente.
- **Licencia: Apache-2.0** (según GitHub API).

Nivel de prioridad: Helix-primary.